ENGINEERING REFERENCE

FO

FORMAOS
COMPLIANCE, SIMPLIFIED

CTO HANDOVER PACK · 02

Developer Documentation

Architecture, technology stack, repository map, local setup, testing, CI/CD and the conventions you'll need to work safely in the codebase.

PREPARED FOR
Farhad Hussain
Incoming CTO

PREPARED BY
Ejaz Hussain
Founder

DATE
13 June 2026

CLASSIFICATION
Confidential

01 / 06

TECHNOLOGY STACK

Stack & key dependencies

A TypeScript monorepo on the Next.js App Router. Server components and server actions do the heavy lifting; the database is the source of truth with row-level security enforced per tenant.

LAYER	TECHNOLOGY	VERSION	ROLE
Framework	Next.js (App Router)	16.1	SSR, server actions, routing
UI	React + TypeScript	19.2 / 5.9	Components, type safety
Styling	Tailwind CSS	3.4	Design tokens, theming
Data / Auth	Supabase (Postgres)	2.98	DB, auth, storage, RLS
Billing	Stripe	22.1	Subscriptions, trials, webhooks
Background jobs	Trigger.dev	—	Async workflows, exports
Rate limit / cache	Upstash Redis	—	Throttling, operational cache
Validation	Zod	4.3	Runtime schema validation
Observability	Sentry · PostHog · OTel	10.4 / 1.36	Errors, analytics, tracing
Testing	Jest · Playwright	30 / 1.58	Unit, E2E, a11y, visual

REPOSITORY

Repository map

```
app/ # Next.js App Router
  (marketing)/ # public site (own theme)
  app/ # authed product — 47 feature areas
  admin/ # platform / founder console
  api/ # 283 REST routes + webhooks
  auth/ onboarding/ # sign-in, MFA, first-run
lib/ # domain services (the business logic)
  supabase/ # clients incl. org-scoped tenant wrapper
  compliance/ frameworks/ # controls, graph, scoring
  billing/ auth/ provisioning/ # Stripe, sessions, tenant setup
  care/ evidence/ export/ admin/ # domain modules
components/ # shared React UI (ui/ primitives + features)
supabase/migrations/ # 252 SQL migrations (schema + RLS history)
framework-packs/ # 8 JSON control libraries
trigger/ # background jobs
e2e/ __tests__/ tests/ # Playwright, unit, integration
.github/workflows/ # 12 CI pipelines
```

WHERE THE LOGIC LIVES

Treat `lib/` as the heart of the system. Routes and components are thin; the real domain rules — tenancy, compliance scoring, billing, provisioning — live in `lib/` services and are unit-tested there.

03 / 06

ARCHITECTURE

Multi-tenancy & the security model

FormaOS is multi-tenant: every organisation's data lives in shared tables keyed by `organization_id`, isolated by Postgres **Row-Level Security**. Because some server work uses the Supabase service-role key (which bypasses RLS), the codebase adds a second structural guard:

> **CRITICAL CONVENTION · THE ORG-SCOPED CLIENT**
>
> `createSupabaseOrgClient(orgId)` (in `lib/supabase/org-scoped.ts`) wraps the admin client so every query is automatically filtered/stamped by the tenant column, and **any table not registered in `TENANT_TABLE_SCOPES` throws at runtime**. This is deliberate: it forces an explicit decision for each table rather than relying on 800+ hand-written filters. When you add a tenant table, you must register it here — a recently-fixed onboarding bug was caused by exactly this omission.

Request path (every authenticated mutation)

01	→	02	→	03	→	04	→	05
Browser Server component or client fetch / action		**Edge proxy** CSRF origin check on all mutating /api/*		**Route / action** Auth + role guard, Zod validation		**Org-scoped client** Tenant filter enforced structurally		**Postgres + RLS** Second isolation layer at the DB

Auth & access

- Supabase Auth with email, Google OAuth, SAML/SSO, and MFA (TOTP + backup codes).
- Role-aware access (owner / admin / member / viewer) plus delegated platform-admin roles in the admin console.
- Sessions are cookie-based with a tracked-session layer; high-risk admin actions are reason-gated and approval-gated.

Key subsystems at a glance

SUBSYSTEM	LIVES IN	WHAT TO KNOW
Compliance engine	lib/compliance*, framework-packs/	Framework packs (JSON) define controls; evidence + evaluations roll up to a posture score and a node-wire compliance graph.
Provisioning	lib/provisioning/	Tenant + user setup on first login (the bootstrap path — see the blocker).
Billing	lib/billing/	Stripe subscriptions, trials, entitlements; webhook-driven state. Plans gate features via entitlements.
Evidence & vault	lib/evidence/, app/app/vault	File uploads mapped to controls; what auditors sample.
Admin console	app/admin, lib/admin	Founder + delegated admins; customer-360, security ops, releases, audit reads.
Background jobs	trigger/	Data retention, purges, audit-chain anchoring, health snapshots (cron + Trigger.dev).
Observability	sentry.*, lib/monitoring	Sentry errors, PostHog analytics, OpenTelemetry traces.
Backups & DR	Supabase PITR · docs/operations/	7-day point-in-time recovery; RPO 60 min / RTO 4 h; CI-gated monthly restore drills logged to `restore_test_runs`.

DATA RESIDENCY

Application compute and edge run in **Vercel** `syd1` **(Sydney)**; the database, authentication and file storage run in **Supabase** `ap-southeast-1` **(Singapore)**. So customer data is served from AU but stored in SG today — worth stating plainly for AU clients and auditors (NDIS, health). Migrating the database to an AU region is the clean answer if in-country data residency becomes a procurement requirement.

GETTING STARTED

Local setup & testing

Run it locally

```
# 1 · install
npm ci

# 2 · environment (copy and fill secrets)
cp .env.example .env.local

# 3 · dev server
npm run dev

# 4 · typecheck / lint / build
npm run typecheck  ·  npm run lint  ·  npm run build
```

Test suites

COMMAND	SUITE	NOTES
npm run test:coverage	Jest unit + integration	The authoritative unit gate — ~5,400 tests
npm run test:e2e	Playwright end-to-end	Chromium is the release gate; full suite ~30 min
npm run test:e2e:onboarding	First-session flow	Onboarding-specific E2E
npm run test:e2e:accessibility	Axe a11y	Accessibility checks
npm run test:visual	Visual regression	Screenshot diffs

06 / 06

DELIVERY

CI/CD & deployment

Deployment is **Vercel native git integration**: merging to `main` triggers a production build and deploy to the Sydney region. GitHub Actions runs 12 workflows on every PR — build, unit, E2E, accessibility, performance/Lighthouse, security, visual regression and compliance gates.

> **INHERITED CAVEAT · PRE-EXISTING RED GATES**
>
> Three CI gates are **red on `main` independent of any given change**: the GDPR and SOC2 compliance gates, and the live-Supabase RLS contract gate (which intermittently hits a statement timeout against the production DB). They are known and pre-existing — not regressions — but they make every PR look scarier than it is. The real go/no-go signal is the `qa-pipeline` unit gate plus the Chromium E2E gate. Cleaning these up is a tracked backlog item (Document 04).

Branch & release discipline

- Branch off `main`; open a PR; let CI run; merge on green (real gates).
- `main` is currently unprotected — adding branch protection once the red gates are fixed is recommended.
- Migrations are applied to the production Supabase project; keep `supabase/migrations/` the source of truth.
- Rollback: Vercel keeps prior production deployments as rollback candidates.